

通用文档分析助手项目介绍与技术实现

1. 项目概述

通用文档分析助手 是一个基于 **RAG (Retrieval-Augmented Generation)** 架构的智能问答系统，旨在帮助用户快速从企业内部文档（如管理制度、流程文件、操作手册等）中获取准确信息。用户只需上传文档（支持 .docx、.pdf、.txt、.csv、.md 等格式），系统便会自动构建知识库，并允许用户通过自然语言提问，系统结合检索到的文档片段与大型语言模型生成精准答案。同时支持多轮对话、流式输出、模型参数调节等功能。

本项目采用 **LangChain** 框架构建核心逻辑，使用 **Chroma** 作为向量数据库，**阿里云通义千问 (DashScope)** 系列模型作为底层 LLM、Embedding 及 Rerank 服务，前端通过 **Gradio** 快速搭建交互界面，并集成 **RAGAS** 进行自动化评估。

2. 功能特性

- 多格式文档上传**：支持常见办公文档格式，自动识别文件类型并调用对应加载器。
- 智能知识库构建**：上传文档后自动进行文本切分、向量化、索引，并存储到 Chroma 向量库中。
- 混合检索策略**：结合向量检索 (Embedding) 与 BM25 关键字检索，再通过重排序模型提升检索质量。
- 对话式问答**：支持多轮对话历史，上下文感知；可切换普通聊天模式（无知识库）与 RAG 模式。
- 流式输出**：实时显示大模型生成内容，提升用户体验。
- 模型参数可调**：支持选择不同 LLM（如 qwen-max、deepseek-r1 等），调节最大输出长度和温度。
- 一键评估**：内置 RAGAS 评估脚本，可对回答的忠实度、上下文相关性、答案相关性等指标进行量化分析。

3. 技术栈

类别	技术/库
前端	Gradio
核心框架	LangChain (Classic, Community, Core)
向量数据库	Chroma
检索增强	EnsembleRetriever, ContextualCompressionRetriever, RePhraseQueryRetriever
大模型服务	阿里云 DashScope (通义千问系列: qwen-max, deepseek-r1/v3, gte-rerank)
文档处理	Unstructured, PyPDF, python-docx, csv, markdown 等
评估工具	RAGAS, datasets
日志	Loguru
其他	SQLite (记录管理), BM25 (rank-bm25)

4. 模块详解

4.1 文档加载与切分 (custom_loader.py)

自定义加载器 `MyCustomLoader` 根据文件扩展名自动选择合适的 LangChain 文档加载器，并使用 `RecursiveCharacterTextSplitter` 将文档切分为固定大小（500 字符）且带有重叠（200 字符）的文本块，保证上下文连贯性。

```
loader_class, params = self.file_type[detect_filetype(file_path)]
self.loader = loader_class(file_path, **params)
self.text_splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", " ", ""],
    chunk_size=500,
    chunk_overlap=200,
)
```

4.2 知识库构建与管理 (knowledge.py)

核心类 `MyKnowledge` 负责知识库的加载、索引和检索器生成。

- **上传文件：** `upload_knowledge` 将用户上传的文件复制到 `./chroma/knowledge/` 目录，并刷新知识库下拉列表。
- **加载知识库：** `load_knowledge` 遍历知识库目录，对每个文件计算 MD5 作为 collection 名称，若尚未建立索引则调用 `create_indexes` 进行索引。
- **创建索引：** `create_indexes` 函数执行以下步骤：
 1. 初始化 Chroma 向量库（指定 collection 和 embedding 模型）。
 2. 初始化 SQLRecordManager 记录文档索引状态，避免重复索引。
 3. 调用 `index` 方法将文档存入向量库（`cleanup="full"` 确保与记录一致）。
 4. 构建混合检索器 `EnsembleRetriever`：结合向量检索（`db.as_retriever(k=3)`）和 BM25 检索（`BM25Retriever.from_documents()`），权重各 0.5。
- **获取检索器：** `get_retrievers` 在已有集合的基础上，进一步包装多层检索增强组件：
 - `RePhraseQueryRetriever`：利用 LLM 对用户问题重述，提取关键信息。
 - `LLMChainFilter`：用 LLM 判断检索到的文档是否与问题相关，过滤不相关文档。
 - `ContextualCompressionRetriever` 嵌套前两者，构成压缩检索器。
 - 最后再套一层 `ContextualCompressionRetriever`，使用阿里重排序模型 `DashScopeRerank` 对候选文档重新排序，取 top 3 作为最终上下文。

这种多级优化确保了检索结果的高度相关性。

4.3 对话与生成 (combine_client.py)

`CombineClient` 继承 `MyKnowledge`，负责与 LLM 交互并管理对话历史。

- **聊天历史：** 使用 `ChatMessageHistory` 存储消息，自动截断至最近 6 条（3 轮对话）。
- **链构建：** `get_chain` 方法根据是否选择知识库，创建不同的处理链：
 - 有知识库：创建 `create_stuff_documents_chain`（将检索到的文档拼接到提示词中）和 `create_retrieval_chain`，组合成 RAG 链。

- 无知识库：直接使用普通提示词模板 + LLM + 自定义流式解析器 `streaming_parse`。
- **历史集成**：通过 `RunnableWithMessageHistory` 将聊天历史自动注入提示词的 `{chat_history}` 占位符。
- **流式输出**：`stream` 方法返回生成器的迭代块，由前端逐步渲染。

4.4 前端界面（`main.py`）

基于 Gradio 构建简洁的 Web UI，包含：

- 模型选择下拉框（通义千问、DeepSeek 系列）。
- 聊天机器人组件，支持复制按钮。
- 参数调节滑块（最大长度、温度）。
- 知识库下拉菜单（动态更新）。
- 文件上传组件（自动触发索引构建）。
- 清除历史按钮。
- 输入文本框和提交按钮。

通过事件绑定实现交互逻辑：用户输入 → `submit_show` 将问题加入历史 → `llm_reply` 流式调用模型，逐块更新聊天内容。文件上传后自动调用 `upload_knowledge`，切换知识库时清空历史。

4.5 模型客户端封装（`models.py`）

提供统一的函数获取 LangChain 封装的阿里云模型客户端：

- `get_lc_model_client`：返回 `ChatOpenAI` 实例（兼容阿里 DashScope）。
- `get_ali_embeddings`：返回 `DashScopeEmbeddings`。
- `get_ali_rerank`：返回 `DashScopeRerank`。

支持从环境变量 `DASHSCOPE_API_KEY` 读取密钥。

4.6 RAG 评估（`rags_eval.py`）

使用 RAGAS 库对系统回答进行自动化评估。示例中针对“请假流程”问题，准备了真实答案（ground truth），构建数据集后计算四个指标：

- `context_precision`：检索到的上下文与问题的相关性。
- `context_recall`：真实答案中的信息是否被检索上下文覆盖。
- `faithfulness`：答案是否忠实于上下文。
- `answer_relevancy`：答案与问题的相关性。

评估结果可帮助开发者量化系统性能，指导调优。

5. 关键实现技术点

5.1 混合检索与重排序

单一检索方式（如仅向量检索）可能遗漏关键词匹配的重要信息。本项目采用 **EnsembleRetriever** 融合向量检索（语义相似）和 BM25（词项匹配），再通过 **DashScopeRerank** 对初步检索结果进行精细排序，极大提升了召回质量。同时，**RePhraseQueryRetriever** 和 **LLMChainFilter** 分别对问题和文档进行二次处理，进一步过滤噪音。

5.2 流式输出与对话历史管理

利用 LangChain 的 `stream` 方法和自定义生成器，实现逐字输出，提升用户体验。通过 `RunnableWithMessageHistory` 自动管理会话历史，简化代码复杂度。

5.3 文档索引与增量更新

使用 `SQLRecordManager` 记录已索引的文档 ID，调用 `index` 方法时自动对比已有记录，实现增量更新，避免重复计算。`cleanup="full"` 确保向量库与记录严格一致。

5.4 多格式文档解析

借助 `Unstructured` 及其生态库，系统能够处理多种格式，且对不同格式采用专门的加载器（如 PDF 用 `PyPDFLoader`，Word 用 `UnstructuredWordDocumentLoader`），提高解析准确性。

5.5 模块化与可扩展性

代码结构清晰，将文档加载、知识库管理、对话逻辑、模型配置分离，便于后续扩展新功能（如支持更多文件类型、替换不同 LLM 平台等）。

6. 环境配置与运行

6.1 安装依赖

bash

```
pip install -r requirements.txt
```

6.2 设置环境变量

在系统环境变量或 `.env` 文件中添加：

```
DASHSCOPE_API_KEY=your_api_key_here
```

6.3 启动应用

bash

```
python main.py
```

访问 Gradio 输出的本地 URL（通常为 `http://127.0.0.1:7860`）即可使用。

6.4 运行评估

修改 `rag_eval.py` 中的问题和真实答案，执行：

bash

```
python rag_eval.py
```

将输出各项指标分数。

7. 总结

本项目实现了一个功能完备的文档问答系统，充分利用了 LangChain 生态的灵活性和阿里云模型的能力，展示了 RAG 技术在企业知识管理中的典型应用。通过多级检索优化、对话历史支持、友好的交互界面以及量化评估工具，为构建生产级文档助手提供了坚实的基础。